

Prompt Tuning

学号：3220100975

姓名：汪珉凯

1. Project Introduction

1.1. 选题

本实验旨在探索提示学习（Prompt Learning）在自然语言处理任务中的应用，具体实现硬提示学习（Hard Prompt Tuning）和软提示学习（Soft Prompt Tuning）两种方法，并在情感分类任务中进行效果对比。

1.2. 工作简介

1.2.1 执行GLUE数据集上的软提示训练代码

通过MindSpore框架运行软提示代码完成训练，并进行评估。

1.2.2 在情感分类任务上实现硬提示与软提示训练

通过MindSpore框架实现硬提示和软提示学习方法，完成情感分类任务（二分类：正面/负面）。具体包括：

- 数据预处理：对情感分类数据集（rt-polarity.meg和rt-polarity.pos）进行清理、分词、索引化和张量化。
- 硬提示学习：设计固定的自然语言模板（如“这个句子的情感是：”）作为输入提示，训练模型。
- 软提示学习：通过可学习的嵌入向量（soft prompts）作为输入提示，利用PEFT库实现微调。
- 测试与评估：比较两种方法在情感分类任务中的表现差异。

1.3. 开发环境及系统运行要求

1.3.1. 开发工具

- MindSpore框架
- Python
- Jupyter Notebook

1.3.2. 开发包

- mindspore
- mindnlp
- numpy
- tqdm
- evaluate

2. Technical Details

2.1. 理论知识

1. 提示学习 (Prompt Learning)

- **核心思想**: 通过设计输入提示 (Prompt) 引导预训练语言模型 (PLM) 生成特定任务的输出, 减少对模型参数的修改。
- **数学表达**:
 - 给定样本 (x_i, y_i) , 通过提示模板 P 将其转换为 $P(x_i)$, 并拼接可学习的软提示嵌入 θ 。
 - 目标函数:

$$\arg \min_{\theta} \mathcal{L}_{\mathcal{D}_r}(\theta) = \arg \max_{\theta} \sum_{(x_i, y_i)} \log P(V(y_i) | \theta; P(x_i))$$

其中 $V(y_i)$ 为标签映射到词汇表的标记, θ 为软提示嵌入参数。

2. 硬提示学习 (Hard Prompt Tuning)

- **特点**: 使用固定的自然语言模板 (如“情感是正面的吗?”) 作为提示。
- **示例**:
 - 原始输入: [CLS] 电影剧情紧凑, 非常精彩。 [SEP]
 - 添加硬提示后: [CLS] 情感是正面的吗? 电影剧情紧凑, 非常精彩。 [SEP]
- **局限性**: 依赖人工设计提示, 泛化能力较弱。

3. 软提示学习 (Soft Prompt Tuning)

- **特点**: 在输入嵌入层前添加可训练的连续向量 (soft prompts), 通过梯度下降优化。
- **示例**:

- 原始嵌入: $[\vec{x}_{cls}, \vec{x}_1, \dots, \vec{x}_m, \vec{x}_{sep}]$
- 添加软提示后: $[\vec{e}_1, \dots, \vec{e}_n, \vec{x}_{cls}, \vec{x}_1, \dots, \vec{x}_{sep}]$

- **优势**: 无需人工设计提示, 自动适应任务需求, 参数效率高。

2.2. 具体算法

本实验主要基于RoBERTa-large模型, 结合软提示 (Soft Prompt) 和硬提示 (Hard Prompt) 两种方法进行文本分类任务 (情感分析)。以下是两种提示方法的具体算法:

软提示 (Soft Prompt)

- 模型架构**: 在RoBERTa-large模型的基础上, 通过PEFT (Parameter-Efficient Fine-Tuning) 库添加可训练的虚拟令牌 (Virtual Tokens)。这些虚拟令牌的嵌入向量在训练过程中会被优化, 而RoBERTa的主干参数保持冻结。
- 训练过程**:
 - 输入文本通过RoBERTa的Tokenizer处理, 生成input_ids和attention_mask。
 - 虚拟令牌的嵌入向量与输入文本的嵌入向量拼接, 形成新的输入表示。
 - 模型基于拼接后的输入进行前向传播, 计算分类损失。
 - 通过反向传播优化虚拟令牌的嵌入向量和分类头的参数。
- 优化目标**: 最小化交叉熵损失函数, 提升模型在验证集上的准确率和F1分数。

硬提示 (Hard Prompt)

1. **提示设计**: 在输入文本前添加固定的提示前缀（如“This review expresses a positive sentiment. The review is: ”），直接引导模型关注情感分类任务。
2. **训练过程**:
 - 输入文本与硬提示拼接后，通过Tokenizer处理生成 - 模型基于拼接后的输入进行前向传播，计算分类损失。
 - 通过反向传播优化RoBERTa的所有参数（全参数微调）。
3. **优化目标**: 同样以交叉熵损失函数为目标，通过硬提示显式引导模型学习任务相关特征。

2.3. 技术细节 (重要函数)

2.3.1. 来自于库的重要函数

1. AutoTokenizer.from_pretrained

- **功能**: 加载预训练模型的Tokenizer，支持动态填充方向（padding_side）和镜像源（mirror）。
- **关键参数**:
 - padding_side: 指定填充方向（left或right），确保与模型架构兼容。
 - mirror: 指定模型下载镜像源（如modelscope）。

2. AutoModelForSequenceClassification.from_pretrained

- **功能**: 加载预训练的分类模型，支持返回字典格式的输出。
- **关键参数**:
 - return_dict=True: 确保输出为字典格式，便于后续处理。

3. get_peft_model

- **功能**: 将基础模型转换为PEFT模型，添加可训练的提示参数。
- **关键配置**:

```
peft_config = PromptTuningConfig(task_type="SEQ_CLS",  
num_virtual_tokens=10)
```

指定任务类型为序列分类，并设置虚拟令牌数量为10。

4. AdamW

- **功能**: 实现自适应矩估计优化器，支持权重衰减。
- **关键参数**:
 - params=model.trainable_params(): 仅优化可训练参数（如虚拟令牌和分类头）。

5. get_linear_schedule_with_warmup

- **功能**: 生成带预热的学习率调度器，线性调整学习率。
- **关键参数**:
 - num_warmup_steps: 预热步数，通常设置为总训练步数的6%。

2.3.2. 自己编写的重要函数

1. MovieReview类

- 功能：处理影评数据集，支持数据读取、清洗、分割和向量化。
- 关键方法：
 - `read_data`：读取文件内容，清洗文本（去除标点、数字等），并根据文件类型（`.pos/.neg`）添加硬提示。
 - `split_dataset`：按比例分割训练集和测试集，确保数据分布均衡。

2. Generator类

- 功能：将数据列表转换为MindSpore的GeneratorDataset兼容格式。
- 关键方法：
 - `__getitem__`：返回数据、标签和索引的三元组，支持动态批处理。

3. MapFunc类

- 功能：将原始文本映射为模型输入（`input_ids`和`attention_mask`）。
- 关键逻辑：

```
outputs = tokenizer(data, truncation=True, max_length=None)
return outputs['input_ids'], outputs['attention_mask'], label
```

4. forward_fn与训练循环

- 功能：定义前向传播和训练流程。
- 关键步骤：
 - 使用`grad_fn`计算损失和梯度。
 - 调用`optimizer.step`和`lr_scheduler.step`更新参数和学习率。
 - 在验证集上计算准确率和F1分数：

```
metric.add_batch(predictions=predictions,
references=references)
eval_metric = metric.compute()
```

3. Experiment Results

3.1. 执行GLUE数据集上的软提示训练代码

- 上传`roberta_sequence_classification.ipynb`文件。
- 环境设置与依赖安装
 - 安装MindNLP、MindSpore和transformers等必要库

○ 设置Hugging Face镜像端点

```

❗ Looking in indexes: http://pip.modelarts.private.com:8888/repository/pypi/simple
Collecting mindnlp==0.4.0
  Downloading http://pip.modelarts.private.com:8888/repository/pypi/packages/mindnlp/0.4.0/mindnlp-0.4.0-py3-none-any.whl (8.4 MB)
    |██████████| 8.4 MB 60.1 MB/s eta 0:00:01
Collecting evaluate
  Downloading http://pip.modelarts.private.com:8888/repository/pypi/packages/evaluate/0.4.3/evaluate-0.4.3-py3-none-any.whl (84 kB)
    |██████████| 84 kB 46.3 MB/s eta 0:00:01
Collecting pyctcdecode
  Downloading http://pip.modelarts.private.com:8888/repository/pypi/packages/pyctcdecode/0.5.0/pyctcdecode-0.5.0-py2.py3-none-any.whl (39 kB)
Requirement already satisfied: sentencepiece in /home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages (from mindnlp==0.4.0) (0.2.0)

```

● 模型与数据集加载

○ 加载Roberta-large模型和tokenizer

```

[WARNING] ME(13583:281473852047536,MainProcess):2025-05-07-15:26:29.987.1 [mindspore/run_check/_check_version.py:398] Can not find the
tbe operator implementation(need by mindspore-ascend). Please check whether the Environment Variable PYTHONPATH is set. For details, re
fer to the installation guidelines: https://www.mindspore.cn/install
/home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages/numpy/core/getlimits.py:499: UserWarning: The value of the smallest
subnormal for <class 'numpy.float64'> type is zero.
  setattr(self, word, getattr(machar, word).flat[0])
/home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages/numpy/core/getlimits.py:89: UserWarning: The value of the smallest
ubnormal for <class 'numpy.float64'> type is zero.
  return self._float_to_str(self.smallest_subnormal)
/home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages/numpy/core/getlimits.py:499: UserWarning: The value of the smallest
subnormal for <class 'numpy.float32'> type is zero.
  setattr(self, word, getattr(machar, word).flat[0])
/home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages/numpy/core/getlimits.py:89: UserWarning: The value of the smallest
ubnormal for <class 'numpy.float32'> type is zero.
  return self._float_to_str(self.smallest_subnormal)
/home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update jup
yter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
Building prefix dict from the default dictionary ...

```

```

100%|██████████| 482/482 [00:00<00:00, 631kB/s]
100%|██████████| 878k/878k [00:00<00:00, 37.1MB/s]
100%|██████████| 446k/446k [00:00<00:00, 40.5MB/s]
100%|██████████| 1.29M/1.29M [00:00<00:00, 43.7MB/s]
/home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages/mindnlp/transformers/tokenization_utils_base.py:1526: FutureW
arning: `clean_up_tokenization_spaces` was not set. It will be set to `True` by default. This behavior will be depracted, and wi
ll be then set to `False` by default.
  warnings.warn(

```

○ 加载GLUE-MRPC数据集并进行预处理

```

datasets = load_dataset("glue", task)
print(next(datasets['train'].create_dict_iterator()))

Downloading readme: 35.3kB [00:00, 516kB/s]
Downloading data: 100%|██████████| 649k/649k [00:02<00:00, 272kB/s]
Downloading data: 100%|██████████| 75.7k/75.7k [00:00<00:00, 94.0kB/s]
Downloading data: 100%|██████████| 308k/308k [00:01<00:00, 288kB/s]
Generating train split: 100%|██████████| 3668/3668 [00:00<00:00, 244574.38 examples/s]
Generating validation split: 100%|██████████| 408/408 [00:00<00:00, 104941.19 examples/s]
Generating test split: 100%|██████████| 1725/1725 [00:00<00:00, 325205.61 examples/s]
{'sentence1': Tensor(shape=[], dtype=String, value='Amrozi accused his brother , whom he called " the witness " , of deliberately dist
orting his evidence .'), 'sentence2': Tensor(shape=[], dtype=String, value='Referring to him as only " the witness " , Amrozi accused
his brother of deliberately distorting his evidence .'), 'label': Tensor(shape=[], dtype=Int64, value= 1), 'idx': Tensor(shape=[], dtyp
e=Int64, value= 0)}

```

可以看到，打印出第一批数据，符合预期：

```

16] print(next(train_dataset.create_dict_iterator()))

Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Default to no trunca
tion.
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Default to no trunca
tion.
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Default to no trunca
tion.
{'input_ids': Tensor(shape=[32, 70], dtype=Int64, value=
[[ 0, 10127, 1001 ... 1, 1, 1],
 [ 0, 975, 26802 ... 1, 1, 1],
 [ 0, 1213, 56 ... 1, 1, 1],
 ...
 [ 0, 133, 1154 ... 1, 1, 1],
 [ 0, 12667, 8423 ... 1, 1, 1],
 [ 0, 32478, 1033 ... 1, 1, 1]]), 'attention_mask': Tensor(shape=[32, 70], dtype=Int64, value=
[[1, 1, 1 ... 0, 0, 0],
 [1, 1, 1 ... 0, 0, 0],
 [1, 1, 1 ... 0, 0, 0],
 ...
 [1, 1, 1 ... 0, 0, 0],
 [1, 1, 1 ... 0, 0, 0],
 [1, 1, 1 ... 0, 0, 0]]), 'labels': Tensor(shape=[32], dtype=Int64, value= [1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1,
1, 1, 0, 1,
1, 1, 0, 0, 1, 1, 1, 0])}

```

● Prompt Tuning配置

○ 设置Prompt Tuning参数：选择任务类型为序列分类，学习率为0.001

```

# peft config
peft_config = PromptTuningConfig(task_type="SEQ_CLS", num_virtual_tokens=10)
# learning rate
lr = 1e-3

```

- 创建Peft模型（参数高效微调模型）
- 模型训练
 - 使用AdamW优化器和线性学习率调度器

```
optimizer = AdamW(params=model.trainable_params(), lr=lr)

# Instantiate scheduler
lr_scheduler = get_linear_schedule_with_warmup(
    optimizer=optimizer,
    num_warmup_steps=0.06 * (len(train_dataset) * num_epochs),
    num_training_steps=(len(train_dataset) * num_epochs),
)
```

打印参与微调的模型参数如下：

```
(Tensor(shape=[1024, 1024], dtype=Float32, value=
[[[-1.10120224e-02, -7.24164350e-03, 2.08059195e-02 ... -2.52948366e-02, -3.65832378e-03, -2.77522877e-02],
 [-1.67993642e-02, 8.08002613e-03, 1.52581530e-02 ... 1.24683203e-02, -1.58226676e-02, -1.18053947e-02],
 [-2.39565764e-02, 7.72664696e-03, -8.20640009e-03 ... -3.20920981e-02, -1.16326455e-02, 7.54686631e-03],
 ...
 [ 1.30826728e-02, -3.10736913e-02, -9.38172976e-04 ... 6.21120026e-03, -2.92483922e-02, -8.80478509e-03],
 [-2.11544195e-03, -7.01228296e-03, 3.13920006e-02 ... 1.15856519e-02, 6.86861342e-03, -1.40164504e-02],
 [ 1.87276164e-03, 1.96019001e-02, -2.25031208e-02 ... -1.84473358e-02, -1.96737014e-02, -1.23416707e-02]]),
 Tensor(shape=[1024], dtype=Float32, value= [ 0.00000000e+00, 0.00000000e+00, 0.00000000e+00 ... 0.00000000e+00, 0.00000000e+00, 0.00000000e+00]),
 Tensor(shape=[2, 1024], dtype=Float32, value=
[[-1.11369826e-02, -2.16085855e-02, -3.27739306e-02 ... -1.71651337e-02, -1.59545820e-02, 2.91206688e-03],
 [-1.10406475e-02, -1.52601255e-02, -2.39929836e-02 ... -3.19496728e-02, -3.55882123e-02, 2.13634558e-02]]),
 Tensor(shape=[2], dtype=Float32, value= [ 0.00000000e+00, 0.00000000e+00]),
 Tensor(shape=[10, 1024], dtype=Float32, value=
[[-1.64303267e+00, -4.11442071e-02, 1.66713998e-01 ... 1.89258671e+00, 8.09491396e-01, -3.04906398e-01],
 [ 8.29266943e-03, -4.73460734e-01, 3.03660691e-01 ... 1.41440964e+00, 4.58577365e-01, -7.52088845e-01],
 [-1.32719779e+00, -2.39679217e-02, 5.54890037e-01 ... -7.16591537e-01, -2.95229495e-01, 1.53800273e+00],
 ...
 [ 7.72343576e-01, 8.29812765e-01, 3.83401871e-01 ... -3.62830877e-01, 9.11299944e-01, 3.96650612e-01],
 [ 1.97685242e+00, 1.52797341e+00, 1.07135057e+00 ... -1.48042589e-01, 5.16637862e-01, -2.50438675e-02],
 [ 1.62037611e+00, 9.98204499e-02, 1.92857289e+00 ... -1.86040664e+00, -1.27325881e+00, -9.82174397e-01]]))
```

- 定义训练逻辑
- 评估指标计算（准确率和F1分数） 首先定义前向计算和梯度计算函数，然后在每个训练周期中遍历训练数据集计算损失和梯度并用优化器更新参数，同时使用学习率调度器调整学习率；在评估阶段，模型切换到评估模式并在验证集上计算预测结果和评估指标（如准确率）。整个过程通过进度条显示迭代进度，并定期输出模型在验证集上的性能指标，用于监控训练效果。

```
0%|          | 0/115 [00:00<?, ?it/s]
...
1%|          | 1/115 [00:41<1:18:46, 41.46s/it]
..
99%|██████████| 114/115 [01:33<00:00, 5.05it/s]
.
100%|██████████| 115/115 [01:35<00:00, 1.20it/s]
100%|██████████| 13/13 [00:04<00:00, 3.16it/s]
epoch 0: {'accuracy': 0.6911764705882353, 'f1': 0.811377245508982}
100%|██████████| 115/115 [00:27<00:00, 4.22it/s]
100%|██████████| 13/13 [00:01<00:00, 7.48it/s]
epoch 1: {'accuracy': 0.7009803921568627, 'f1': 0.7993421052631579}
100%|██████████| 115/115 [00:29<00:00, 3.95it/s]
100%|██████████| 13/13 [00:01<00:00, 7.64it/s]
epoch 2: {'accuracy': 0.7132352941176471, 'f1': 0.821917808219178}
100%|██████████| 115/115 [00:28<00:00, 4.05it/s]
100%|██████████| 13/13 [00:01<00:00, 7.58it/s]
epoch 3: {'accuracy': 0.7034313725490197, 'f1': 0.8158295281582952}
100%|██████████| 115/115 [00:26<00:00, 4.32it/s]
100%|██████████| 13/13 [00:01<00:00, 8.02it/s]
epoch 4: {'accuracy': 0.7034313725490197, 'f1': 0.8163884673748103}
```

3.2 在情感分类任务上实现软提示训练

- 安装并导入相关依赖库，主要包括mindspore, mindnlp等，以及后者自带的peft包。

- 配置超参数以及`peft`

```
from easydict import EasyDict as edict
cfg = edict({
    'model_name': "AI-ModelScope/roberta-large",
    'batch_size': 64,
    'epoch_size': 10,
    'data_path': './data/',
    'device_target': 'Ascend',
    'lr': 1e-3,
    'task': "mrpc"
})

mindspore.set_context(device_target=cfg.device_target)
```

可以看到，我将`batch`大小设置为64，学习率设置为`1e-3`，`epochs`设置为10，以及一系列其他参数，并规定环境为`Ascend`。

- 定义`tokenizer`，用于数据格式转化。

`tokenizer`

```
# load tokenizer
if any(k in cfg.model_name for k in ("gpt", "opt", "bloom")):
    padding_side = "left"
else:
    padding_side = "right"

tokenizer = AutoTokenizer.from_pretrained(cfg.model_name, padding_side=padding_side, mirror="modelscope")
if getattr(tokenizer, "pad_token_id") is None:
    tokenizer.pad_token_id = tokenizer.eos_token_id
```

/home/ma-user/anaconda3/envs/MindSpore/lib/python3.9/site-packages/mindnlp/transformers/tokenization_utils_base.py:1526: FutureWarning: `clean\_up\_tokenization\_spaces` was not set. It will be set to `True` by default. This behavior will be deprecated, and will be then set to `False` by default.
warnings.warn()

- 后面根据TextCNN等内容进行数据处理：
- 定义`Generator`类

```
class Generator():
    def __init__(self, input_list):
        self.input_list=input_list
    def __getitem__(self,item):
        return (self.input_list[item][0],
                np.array(self.input_list[item][1],dtype=np.int32),item) # 返回数据与标签
    def __len__(self):
        return len(self.input_list)
```

- 类功能：
 - 用于封装数据集，提供索引访问和长度计算的功能。
- 主要方法：
 - `__getitem__(self, item)`：根据索引返回一个数据样本，包括数据、标签和索引。
 - `__len__(self)`：返回数据集的长度。
- 定义`MovieRiview`类

- 定义 `read_data` 函数

```
def read_data(self, filePath):

    with open(filePath, 'r') as f:

        for sentence in f.readlines():
            sentence = sentence.replace('\n', '')\
                               .replace('"', '')\
                               .replace('\\', '')\
                               .replace('.', '')\
                               .replace(',', '')\
                               .replace('[', '')\
                               .replace(']', '')\
                               .replace('(', '')\
                               .replace(')', '')\
                               .replace(':', '')\
                               .replace('--', '')\
                               .replace('-', ' ')\
                               .replace('\\', '')\
                               .replace('0', '')\
                               .replace('1', '')\
                               .replace('2', '')\
                               .replace('3', '')\
                               .replace('4', '')\
                               .replace('5', '')\
                               .replace('6', '')\
                               .replace('7', '')\
                               .replace('8', '')\
                               .replace('9', '')\
                               .replace('`', '')\
                               .replace('=', '')\
                               .replace('$', '')\
                               .replace('/', '')\
                               .replace('*', '')\
                               .replace('; ', '')\
                               .replace('<b>', '')\
                               .replace('%', '')

            if sentence:
                self.word_num += len(sentence)
                self.maxlen = self.maxlen if self.maxlen >= len(sentence) else len(sentence)
                self.minlen = self.minlen if self.minlen <= len(sentence) else len(sentence)
                if 'pos' in filePath:
                    self.Pos.append([sentence, self.feelMap['pos']])
                else:
                    self.Neg.append([sentence, self.feelMap['neg']])
```

- 定义 `train_dataset` 函数

```
def create_train_dataset(self):
    dataset = ds.GeneratorDataset(source=Generator(input_list=self.train), column_names=["data", "label", "idx"], shuffle=False)
    return dataset

def create_test_dataset(self):
    dataset = ds.GeneratorDataset(source=Generator(input_list=self.test), column_names=["data", "label", "idx"], shuffle=False)
    return dataset
```

- 类功能：

- 用于加载、预处理和分割影评数据集，并提供训练和测试数据集的生成方法。

- 主要方法和属性：

- `__init__(self, root_dir, split)`:
 - 初始化类，加载影评数据集。
 - 读取影评文件（正面影评和负面影评），并统计句子长度等信息。
 - 将影评数据分为正面（Pos）和负面（Neg）两类。
 - 根据 `split` 参数划分训练集和测试集。
- `read_data(self, filePath)`:
 - 读取影评文件并进行简单的文本清理（去除标点符号、数字等）。
 - 将清理后的句子和对应的标签（正面或负面）存储到 Pos 或 Neg 列表中。
- `text2vec(self, maxlen)`:
 - 将句子转换为固定长度的向量表示。

- 构建词汇表 (Vocab)，将每个单词映射为一个唯一的索引。
- `split_dataset(self, split):`
 - 根据 `split` 参数将数据集划分为训练集和测试集。
 - 使用随机打乱的方式确保数据分布均匀。
- `get_dict_len(self):`
 - 返回词汇表的长度。
- `create_train_dataset(self)` 和 `create_test_dataset(self):`
 - 使用 `GeneratorDataset` 创建 MindSpore 数据集对象，分别用于训练和测试。
- 定义 `MapFunc` 类

```
class MapFunc(BaseMapFunction):
    def __call__(self, data, label, idx):
        outputs = tokenizer(data, truncation=True, max_length=None)
        return outputs['input_ids'], outputs['attention_mask'], label
```

- 定义 `get_dataset` 函数，用于将数据集转换为批量化的模型输入。

```
def get_dataset(dataset, tokenizer):
    input_columns=['data', 'label', 'idx']
    output_columns=['input_ids', 'attention_mask', 'labels']
    dataset = dataset.map(MapFunc(input_columns, output_columns),
                          input_columns, output_columns)
    dataset = dataset.padded_batch(cfg.batch_size, pad_info={'input_ids': (None, tokenizer.pad_token_id),
                                                             'attention_mask': (None, 0)})
```

- 加载影评数据集，并生成训练和测试数据集。

```
instance = MovieReview(root_dir=cfg.data_path, split=0.9)
train_dataset = instance.create_train_dataset()
test_dataset = instance.create_test_dataset()
train_dataset = get_dataset(train_dataset, tokenizer)
test_dataset = get_dataset(test_dataset, tokenizer)
```

结果如下：

```
print(next(train_dataset.create_dict_iterator()))
```

```
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Default to no
truncation.
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Default to no
truncation.
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Default to no
truncation.
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Default to no
truncation.
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Default to no
truncation.
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Default to no
truncation.
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Default to no
truncation.
Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. Default to no
truncation.
{'input_ids': Tensor(shape=[64, 51], dtype=Int64, value=
[[ 0, 627, 2933 ... 1, 1, 1],
 [ 0, 405, 18166 ... 1, 1, 1],
 [ 0, 102, 13363 ... 1, 1, 1],
 ...
 [ 0, 627, 1535 ... 1, 1, 1],
 [ 0, 260, 15955 ... 1, 1, 1],
 [ 0, 405, 429 ... 1, 1, 1]]), 'attention_mask': Tensor(shape=[64, 51], dtype=Int64, value=
[[1, 1, 1 ... 0, 0, 0],
 [1, 1, 1 ... 0, 0, 0],
 [1, 1, 1 ... 0, 0, 0],
 ...
 [1, 1, 1 ... 0, 0, 0],
 [1, 1, 1 ... 0, 0, 0],
 [1, 1, 1 ... 0, 0, 0]]), 'labels': Tensor(shape=[64], dtype=Int32, value= [0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 1,
0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0,
1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1])}
```

- 加载预训练的 RoBERTa 模型，并应用 PEFT 配置

```
# load model
model = AutoModelForSequenceClassification.from_pretrained(cfg.model_name,
return_dict=True,mirror="modelscope")
model = get_peft_model(model, peft_config)
# print number of trainable parameters
model.print_trainable_parameters()
```

```
[WARNING] DEVICE(92155,ffffb3a460b0,python):2025-05-17-18:39:12.576.033 [mindspore/ccsrc/plugin/device/ascend/hal/device/ascend_
vmm_adapter.h:149] CheckVmmDriverVersion] Open file /etc/ascend_install.info failed.
[WARNING] DEVICE(92155,ffffb3a460b0,python):2025-05-17-18:39:12.576.118 [mindspore/ccsrc/plugin/device/ascend/hal/device/ascend_
vmm_adapter.h:188] CheckVmmDriverVersion] Driver version is less than 24.0.0, vmm is disabled by default, drvier_version: 23.0.6
Some weights of RobertaForSequenceClassification were not initialized from the model checkpoint at AI-ModelScope/roberta-large a
nd are newly initialized: ['classifier.dense.bias', 'classifier.dense.weight', 'classifier.out_proj.bias', 'classifier.out_proj.
weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
trainable params: 1,061,890 || all params: 356,423,684 || trainable%: 0.2979291353713745
```

- 定义优化器为AdamW,学习策略为包含warmup阶段的线性学习率调整策略。

```
optimizer = AdamW(params=model.trainable_params(), lr=cfg.lr)

# Instantiate scheduler
lr_scheduler = get_linear_schedule_with_warmup(
    optimizer=optimizer,
    num_warmup_steps=0.06 * (len(train_dataset) * cfg.epoch_size),
    num_training_steps=(len(train_dataset) * cfg.epoch_size)
)
```

- 按照如下步骤定义训练逻辑：
 - 1. 构建正向计算函数
 - 2. 函数变换，获取微分函数
 - 3. 定义训练一个step的逻辑
 - 4. 遍历训练数据集进行模型训练，同时每一个epoch后，遍历验证数据集获取当前的评价指标（accuracy、f1 score）（具体和3.1部分一致，此处不再赘述）

3.2.1 情感分类软提示训练结果

```

100%|██████████| 150/150 [00:56<00:00, 2.66it/s]
.
100%|██████████| 17/17 [00:03<00:00, 5.12it/s]
epoch 0: {'accuracy': 0.662605435801312, 'f1': 0.7268588770864947}
100%|██████████| 150/150 [00:44<00:00, 3.39it/s]
100%|██████████| 17/17 [00:02<00:00, 6.56it/s]
epoch 1: {'accuracy': 0.7666354264292409, 'f1': 0.7774798927613941}
100%|██████████| 150/150 [00:44<00:00, 3.39it/s]
100%|██████████| 17/17 [00:02<00:00, 6.38it/s]
epoch 2: {'accuracy': 0.7666354264292409, 'f1': 0.7880851063829787}
100%|██████████| 150/150 [00:44<00:00, 3.39it/s]
100%|██████████| 17/17 [00:02<00:00, 6.19it/s]
epoch 3: {'accuracy': 0.8059981255857545, 'f1': 0.8092165898617512}
100%|██████████| 150/150 [00:44<00:00, 3.39it/s]
100%|██████████| 17/17 [00:02<00:00, 6.13it/s]
epoch 4: {'accuracy': 0.8031865042174321, 'f1': 0.8011363636363636}
100%|██████████| 150/150 [00:44<00:00, 3.40it/s]
100%|██████████| 17/17 [00:02<00:00, 6.25it/s]
epoch 5: {'accuracy': 0.7947516401124649, 'f1': 0.8025247971145176}
100%|██████████| 150/150 [00:44<00:00, 3.39it/s]
100%|██████████| 17/17 [00:02<00:00, 6.17it/s]
epoch 6: {'accuracy': 0.8031865042174321, 'f1': 0.8018867924528302}
100%|██████████| 150/150 [00:44<00:00, 3.40it/s]
100%|██████████| 17/17 [00:02<00:00, 6.07it/s]
epoch 7: {'accuracy': 0.8069353327085286, 'f1': 0.8081936685288641}
100%|██████████| 150/150 [00:44<00:00, 3.39it/s]
100%|██████████| 17/17 [00:02<00:00, 6.21it/s]
epoch 8: {'accuracy': 0.8050609184629803, 'f1': 0.8098720292504571}
100%|██████████| 150/150 [00:44<00:00, 3.39it/s]
100%|██████████| 17/17 [00:02<00:00, 6.09it/s]
epoch 9: {'accuracy': 0.8106841611996252, 'f1': 0.8115671641791045}

```

可以看到，随着训练轮数的上升，准确率逐步提升到80%左右趋于稳定，而f1分数则是趋于0.8达到稳定。

3.3 在情感分类任务上实现硬提示训练

基本与软提示的训练相同，但是有以下几点不同：

- 新增`hard_prompt`定义：

新增部分：定义hard_prompt提示部分

```

hard_prompt = {
    "positivePrompt": [
        "This review expresses a positive sentiment. The review is: "
    ],
    "negativePrompt": [
        "This review expresses a negative sentiment. The review is: "
    ]
}

```

Python

- 修改`MovieReview`类的`read_data`函数

```

for sentence in f.readlines():
    if filePath.endswith('.pos'):
        # 影评为正面
        sentence = hard_prompt['positivePrompt'][0] + " ," + sentence
    else:
        # 影评为负面
        sentence = hard_prompt['negativePrompt'][0] + " ," + sentence

```

可以看到：根据file名，在训练数据前加上固定的提示词。

3.3.1 情感分类硬提示训练结果

```
100%|██████████| 150/150 [01:04<00:00, 2.34it/s]
.
100%|██████████| 17/17 [00:03<00:00, 5.44it/s]
epoch 0: {'accuracy': 0.8641049671977507, 'f1': 0.8782535684298909}
100%|██████████| 150/150 [00:51<00:00, 2.93it/s]
100%|██████████| 17/17 [00:02<00:00, 5.88it/s]
epoch 1: {'accuracy': 0.9690721649484536, 'f1': 0.9690721649484536}
100%|██████████| 150/150 [00:51<00:00, 2.93it/s]
100%|██████████| 17/17 [00:02<00:00, 5.74it/s]
epoch 2: {'accuracy': 0.9315838800374883, 'f1': 0.9355692850838482}
100%|██████████| 150/150 [00:51<00:00, 2.92it/s]
100%|██████████| 17/17 [00:02<00:00, 5.93it/s]
epoch 3: {'accuracy': 0.9353327085285849, 'f1': 0.9386666666666666}
100%|██████████| 150/150 [00:51<00:00, 2.92it/s]
100%|██████████| 17/17 [00:02<00:00, 5.80it/s]
epoch 4: {'accuracy': 0.9812558575445174, 'f1': 0.9811676082862524}
```

可以看到，使用硬提示训练后，准确率也能稳步提升。